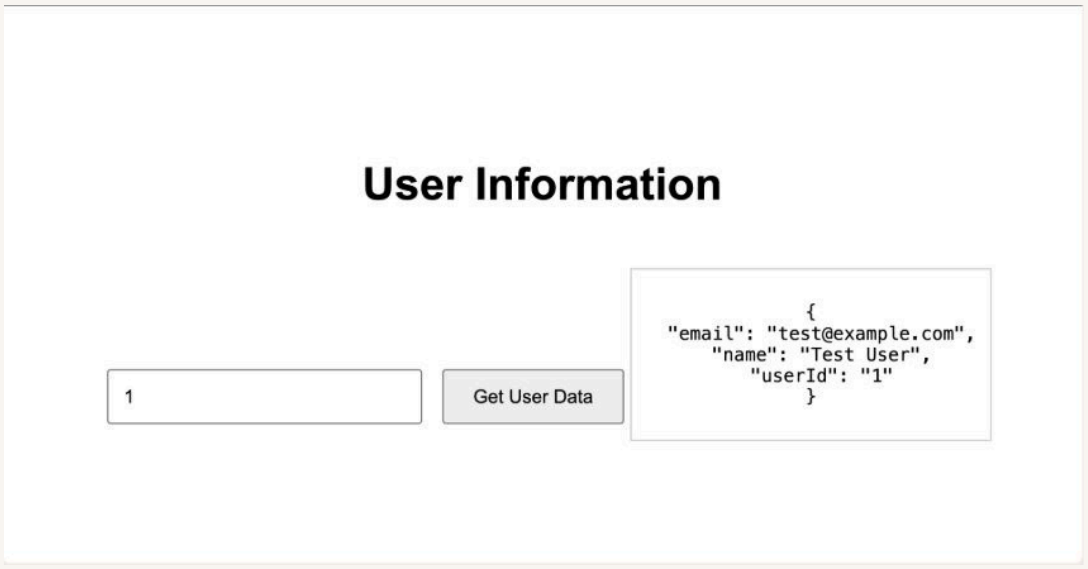
Build a Three-Tier Web App

Mohammed Dawoud Mota



Introducing Today's Project!

In this project, I built a complete three-tier web application on AWS and showed that I understand how each layer works together. My goal is to walk through the full process of setting up the presentation tier with S3 and CloudFront, creating the logic tier using a Lambda function and API Gateway, and finishing with the data tier by storing and retrieving information from DynamoDB. By the end of this project, I want to demonstrate that I can connect all three layers into a functioning architecture that delivers a website, processes requests, and fetches data from a database in a clean, scalable way.

Tools and concepts

In this project, I learnt how different AWS services work together to build a complete three-tier web application. I worked with S3 to store and host my website files, and used CloudFront to distribute the site globally with fast performance. I built the logic tier using AWS Lambda, where my backend code runs without needing any servers. I connected that logic to the outside world using API Gateway, which let my website send requests to my Lambda function. Finally, I used DynamoDB as the data tier to store and retrieve user information. Along the way, I also learnt important concepts like three-tier architecture, serverless computing, REST APIs, and CORS, and how all these pieces fit together to create a scalable, cloud-based application.

Project reflection

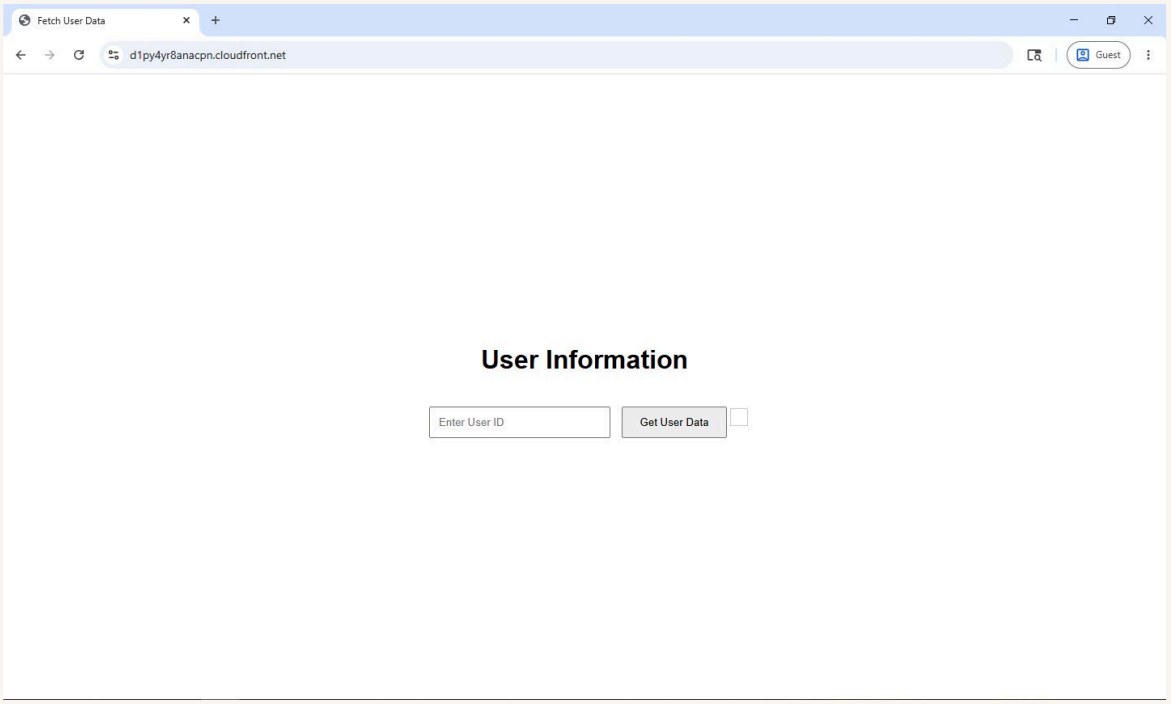
This project took me over a few days to complete, as I was doing each tier on a different day.

I chose to do this project today because I wanted to learn how to create a full functional multi-tier web application on AWS.

Presentation tier

For the presentation tier, I am setting up the part of the application that users actually see and interact with. This includes creating an S3 bucket to store the website files, uploading the HTML, CSS, and JavaScript that make up the front-end, and configuring a CloudFront distribution so the site can be delivered quickly and reliably to users around the world. The goal of this tier is to provide a clean, accessible interface while ensuring that the content loads efficiently and is ready to connect with the deeper layers of the application once the logic and data tiers are in place.

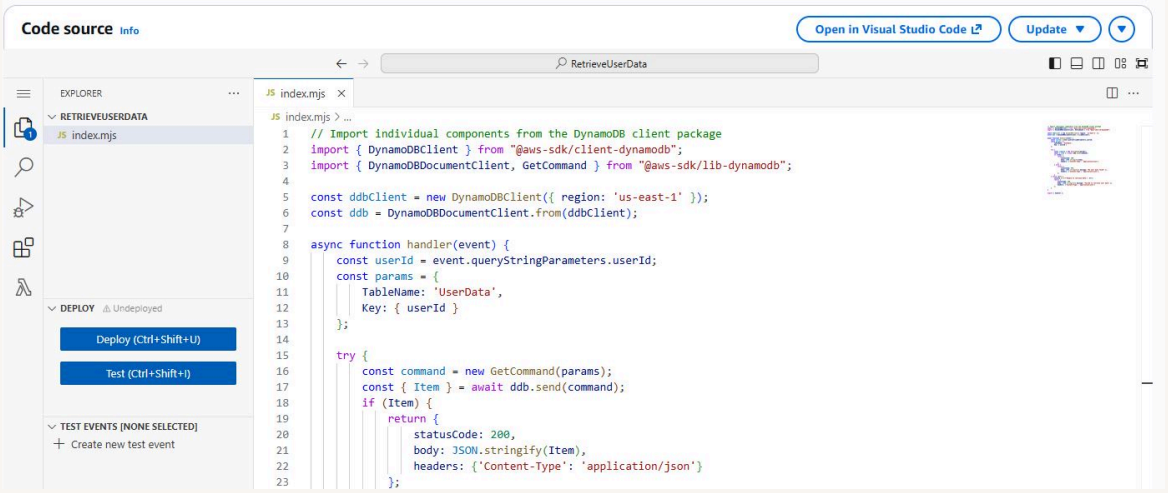
I accessed my website by using the domain name provided by CloudFront after the distribution was created. Once the S3 bucket was set up and the files were uploaded, CloudFront generated a unique distribution URL. I copied that URL from the CloudFront console and opened it in my browser. This allowed me to view the website exactly as a user would, delivered through CloudFront’s global network rather than directly from the S3 bucket.


Fetch User Data
d1py4yr8anacpn.cloudfront.net
Guest
User Information
Enter User ID
Get User Data

Logic tier

In the logic tier, I am setting up the part of the application that handles all of the processing behind the scenes. This includes creating a Lambda function that will retrieve user information from a DynamoDB table and preparing the code that allows it to respond to incoming requests. I am also configuring an API Gateway REST API so that users can interact with the Lambda function through a structured endpoint. Together, these components form the core of the application's backend logic, allowing the front-end to request data and receive meaningful responses in a secure and scalable way.

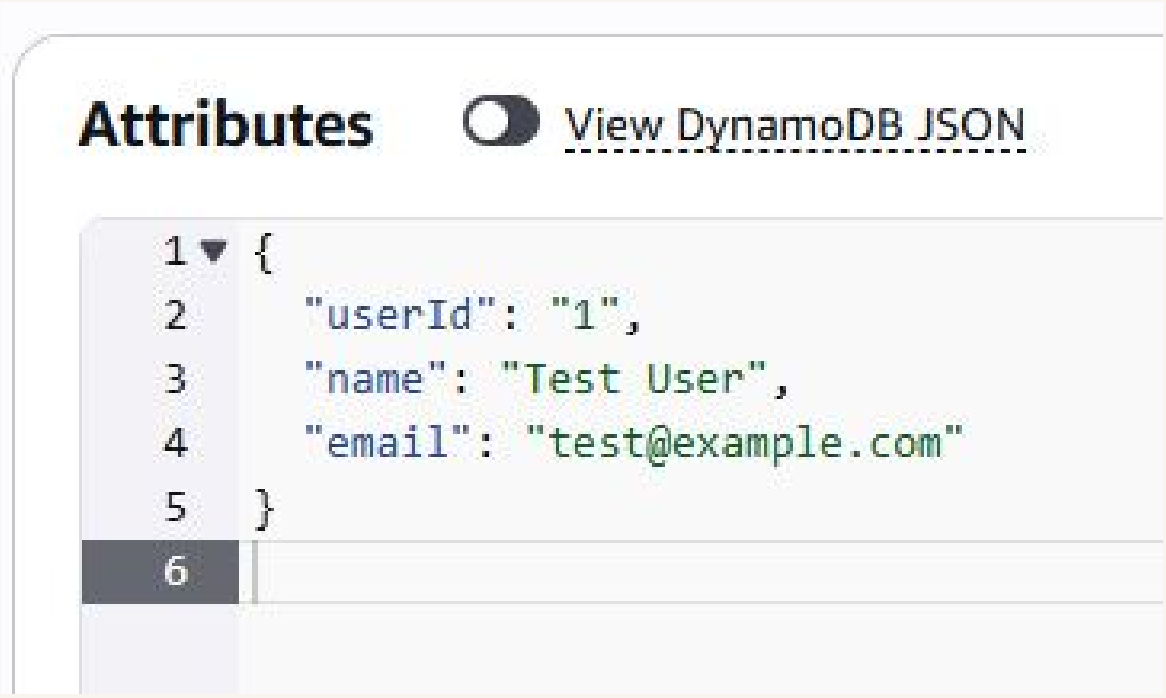
The Lambda function retrieves data by using the AWS SDK to connect directly to the DynamoDB table and query it based on the userId that comes in through the API request. When the function is triggered, it reads the query string parameters from the incoming event, extracts the userId, and builds a request object that specifies the table name and the key it needs to look up. It then sends that request to DynamoDB using the DocumentClient, which returns the matching item if it exists. Once the data is retrieved, the function formats the result into a JSON response and sends it back through API Gateway so the user can see the information returned from the database.


Code source Info
Open in Visual Studio Code
Update
RetrieveUserData
EXPLORER
RETRIEVEUSERDATA
index.mjs
DEPLOY
Undeployed
Deploy (Ctrl+Shift+U)
Test (Ctrl+Shift+I)
TEST EVENTS [NONE SELECTED]
Create new test event
// Import individual components from the DynamoDB client package
import { DynamoDBClient } from "@aws-sdk/client-dynamodb";
import { DynamoDBDocumentClient, GetCommand } from "@aws-sdk/lib-dynamodb";
const ddbClient = new DynamoDBClient({ region: 'us-east-1' });
const ddb = DynamoDBDocumentClient.from(ddbClient);
async function handler(event) {
const userId = event.queryStringParameters.userId;
const params = {
TableName: 'UserData',
Key: { userId }
};
try {
const command = new GetCommand(params);
const { Item } = await ddb.send(command);
if (Item) {
return {
statusCode: 200,
body: JSON.stringify(Item),
headers: {'Content-Type': 'application/json'}
};

Data tier

In the data tier, I am setting up the part of the application responsible for storing and managing the information that the rest of the system relies on. This involves creating a DynamoDB table that will hold user records, defining the partition key that uniquely identifies each item, and adding sample data so the Lambda function has something to retrieve during testing. By establishing this database layer, I’m ensuring that the application has a reliable and scalable place to store user information, which completes the foundation needed for the logic tier to process requests and for the presentation tier to display meaningful results.

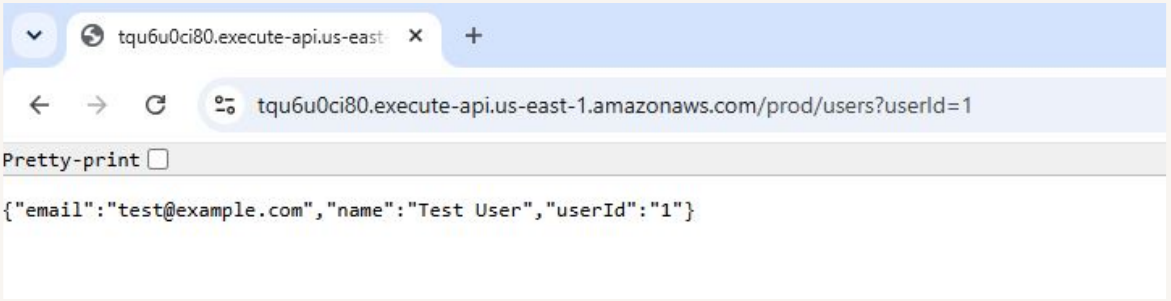
We are using DynamoDB as the database layer of the application to store and manage user information in a fast, flexible, and scalable way. It serves as the central place where each user’s data is kept, identified by a unique partition key so the Lambda function can quickly retrieve the correct record when an API request comes in. By storing the data in DynamoDB, the application gains a reliable backend that can handle growth, respond efficiently to queries, and support the overall three-tier architecture without requiring any traditional server management.


Attributes
View DynamoDB JSON
1 {
2 "userId": "1",
3 "name": "Test User",
4 "email": "test@example.com"
5 }
6

Logic and Data tier

In this step, I am connecting the front-end of the application to the backend by updating the JavaScript code so the website can call the API Gateway endpoint and display the data returned from the Lambda function. This involves adding a fetch request inside the script.js file that sends a request to the API, receives the response, and then updates the webpage with the user information stored in DynamoDB. By doing this, I'm completing the final link between all three tiers and allowing the website to show real, dynamic data instead of static content.

When I tested my API, I used three different approaches to make sure everything was working correctly. I started by testing the Lambda function directly in the Lambda console to confirm that it could read from DynamoDB and return the right data. After that, I tested the API inside API Gateway using the built-in test tool, which let me verify that the request was reaching the Lambda function and that the response format was correct. Finally, I tested the deployed API by opening the Invoke URL in my browser with the userId parameter added. This confirmed that the API was accessible publicly and that the full request flow was working end to end.

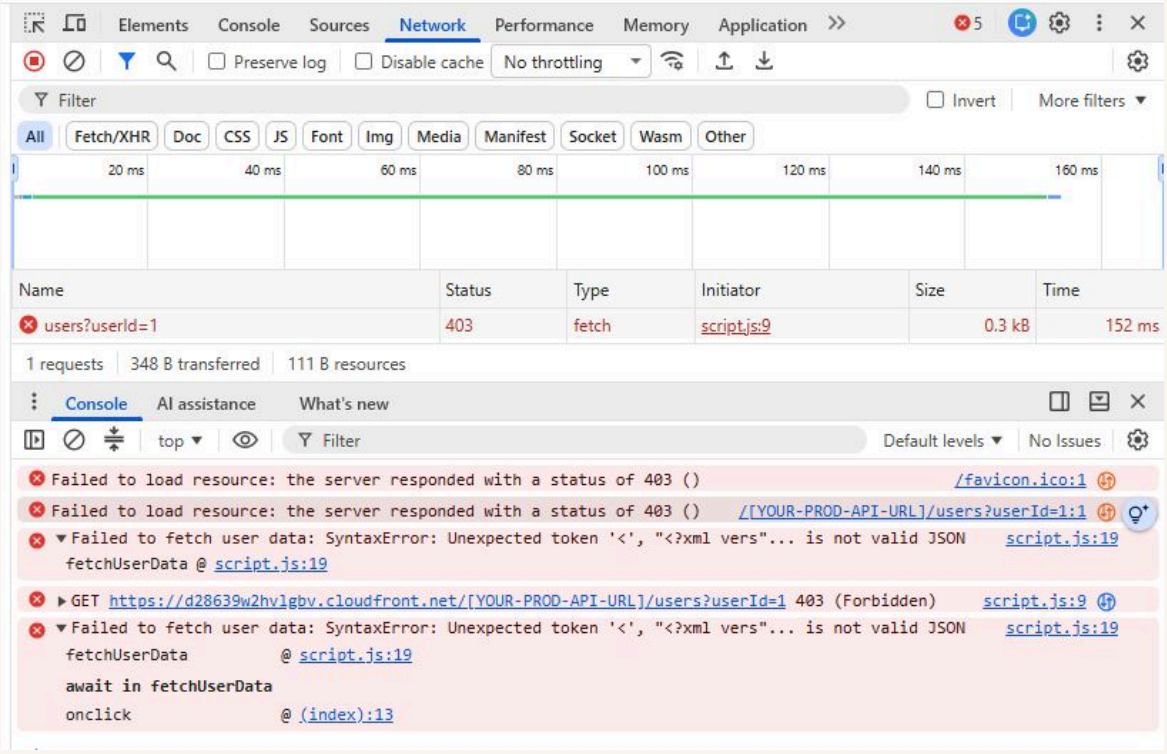

tqu6u0ci80.execute-api.us-east
tqu6u0ci80.execute-api.us-east-1.amazonaws.com/prod/users?userId=1
Pretty-print
{"email":"test@example.com","name":"Test User","userId":"1"}

Console Errors

I got an error on the CloudFront distributed website because the browser was trying to fetch data from my API, but the request wasn't allowed. Even though the API works when I test it directly, the website is being served from a different domain, and the browser blocks that kind of request unless the API has the right CORS settings. Since those settings weren't in place yet, the browser refused to accept the response, which is why no user data showed up on the page.

I uploaded an updated script.js because the original file was still pointing to the placeholder API URL instead of my actual deployed API Gateway endpoint. Since the website relies on that JavaScript file to make the request to the backend, the front-end had no way of reaching my real API until I replaced the placeholder with my own Invoke URL. Updating and re-uploading the file made sure the website was calling the correct endpoint so it could finally retrieve and display the user data.

I ran into another error because even though I had updated the script and connected it to my real API, the browser was still enforcing its own rules when trying to load the data. The website was being served through CloudFront, but the API response didn't include the CORS headers the browser needed in order to accept it. Since those headers weren't there yet, the browser blocked the response again, which caused the error to show up on the page instead of the user data.


Elements
Console
Sources
Network
Performance
Memory
Application
Preserve log
Disable cache
No throttling
Filter
Invert
More filters
All
Fetch/XHR
Doc
CSS
JS
Font
Img
Media
Manifest
Socket
Wasm
Other
20 ms
40 ms
60 ms
80 ms
100 ms
120 ms
140 ms
160 ms
Name
Status
Type
Initiator
Size
Time
users?userId=1
403
fetch
script.js:9
0.3 kB
152 ms
1 requests
348 B transferred
111 B resources
Console
AI assistance
What's new
top
Filter
Default levels
No Issues
Failed to load resource: the server responded with a status of 403 ()
/favicon.ico:1
Failed to load resource: the server responded with a status of 403 ()
/[YOUR-PROD-API-URL]/users?userId=1:1
Failed to fetch user data: SyntaxError: Unexpected token '<', "<?xml vers"... is not valid JSON
script.js:19
fetchUserData @ script.js:19
GET https://d28639w2hvlgbv.cloudfront.net/[YOUR-PROD-API-URL]/users?userId=1 403 (Forbidden)
script.js:9
Failed to fetch user data: SyntaxError: Unexpected token '<', "<?xml vers"... is not valid JSON
script.js:19
fetchUserData @ script.js:19
await in fetchUserData
onclick @ (index):13

Resolving CORS Errors

I just updated the CORS settings in my API so the browser would finally be allowed to accept the response coming from API Gateway. Before this change, the API was returning data correctly, but the browser blocked it because the response didn’t include the headers needed for cross-origin requests. By adding the proper CORS configuration, I made sure the API explicitly allows requests coming from my CloudFront-hosted website, which lets the data flow through without being blocked.

I updated my Lambda function because the API needed to return the correct CORS headers, and those headers have to come directly from the Lambda response when I’m using Lambda proxy integration. Even though I fixed the CORS settings in API Gateway, the browser still wasn’t seeing the headers it needed, so the request kept getting blocked. By updating the Lambda function to include the proper headers in every response, I made sure the browser could finally accept the data coming back from the API.


index.mjs
index.mjs > handler
async function handler(event) {
},
body: JSON.stringify(Item)
};
} else {
return {
statusCode: 404,
headers: {
'Content-Type': 'application/json',
'Access-Control-Allow-Origin': '*'
},
body: JSON.stringify({ message: "No user data found" })
};
}
} catch (err) {
console.error("Unable to retrieve data:", err);
Amazon Q Tip 1/3: Start typing to get suggestions ([
return {
statusCode: 500,
headers: {
'Content-Type': 'application/json',
'Access-Control-Allow-Origin': '*'
},
body: JSON.stringify({ message: "Failed to retrieve user data" })
};
}

Fixed Solution

I verified the fixed solution by going back to my CloudFront URL and testing the website again after updating the API and redeploying it. Once the page loaded, I entered a userId and checked whether the request successfully reached my API Gateway endpoint and returned the correct JSON response. When the user data appeared on the page without any errors, that confirmed that the API method, CORS settings, and Lambda response were all working together the way they were supposed to.